03 - 训练流程面试题

> 共 32 道题，涵盖 Tokenizer、预训练、SFT、LoRA、DPO、PPO、GRPO、知识蒸馏、训练工程等全链路。

Q1: 什么是 Tokenizer？为什么需要 Tokenizer？

详细版：

Tokenizer 将原始文本切分为模型可以处理的最小单元（token），并映射为整数 ID。模型无法直接处理字符串，需要 Tokenizer 作为文本和数字之间的桥梁。

为什么不直接按字符切分？ - 字符级别词表太小（几百个），序列长度会非常长，计算成本高 - 字符缺乏语义，模型需要学习更多层次的组合关系

为什么不直接按词切分？ - 词表会非常大（数十万），Embedding 层参数爆炸 - 无法处理未登录词（OOV）

子词（Subword）Tokenizer 是折中方案：高频词保持完整，低频词拆分为子词片段。

MiniMind 使用 BPE 算法训练了一个 6400 大小的词表，平衡了词表大小和序列长度。

精简版：

Tokenizer 将文本切分为子词 token 并映射为 ID。子词是字符级和词级的折中：高频词完整保留，低频词拆分为子词。MiniMind 用 BPE 训练了 6400 词表。

Q2: BPE（Byte Pair Encoding）算法的原理是什么？

详细版：

BPE 算法步骤：

1. **初始化**：将所有文本拆分为单个字符（或字节），作为初始词表
2. **统计**：计算相邻 token 对的出现频率
3. **合并**：将出现频率最高的 token 对合并为一个新 token，加入词表

4. **重复**：重复步骤 2-3，直到词表达到预设大小

示例：

```
语料: "low lower lowest"
初始: [l, o, w, e, r, s, t, _]
第1轮: 'l'+'o' 最频繁 → 合并为 'lo' → [lo, w, e, r, s, t, _]
第2轮: 'lo'+'w' 最频繁 → 合并为 'low' → [low, e, r, s, t, _]
...
```

优点： - 自底向上构建，高频子串自动成为一个 token - 能处理任何未登录词（拆分到字符/字节级别即可） - 词表大小可控

MiniMind 词表大小 6400 的考量： - 太小：序列过长，训练慢，长距离依赖难学 - 太大：Embedding 参数多，对小模型是负担（6400×768≈4.9M，占 64M 的 7.6%） - 6400 是小模型的合理折中

精简版：

BPE 从字符开始，不断合并最高频的相邻 token 对，直到词表达到目标大小。高频子串自动成为 token，未登录词可回退到字符/字节。MiniMind 的 6400 词表是小模型的合理折中。

Q3: Byte-level BPE 和 Character-level BPE 有什么区别?

详细版：

特性	Character-level BPE	Byte-level BPE
初始单元	Unicode 字符	UTF-8 字节（256 个）
初始词表大小	取决于语料中的字符种类（可能数千）	固定 256
OOV 问题	可能遇到训练时未见过的字符	完全无 OOV（任何文本都可编码）
中文处理	每个汉字是一个字符	每个汉字 3 个字节

Byte-level BPE（GPT-2/GPT-4 使用）： - 保证任何输入都能编码，零 OOV - 中文等非 ASCII 文本的序列长度会更长 - 现代主流方案

实际中很多 Tokenizer（如 SentencePiece）在 BPE 基础上有各种变体和优化。

精简版：

Character-level BPE 以字符为初始单元，可能有 OOV；Byte-level BPE 以 256 个字节为初始单元，完全无 OOV，是主流方案。中文在 Byte-level 下每个汉字约 3 字节。

Q4: 预训练的目标函数是什么？Next Token Prediction 是怎么工作的？

详细版：

预训练使用 **Causal Language Modeling**（因果语言建模），目标是 Next Token Prediction（NTP）：

给定序列 $x_1, x_2, ..., x_n$，最大化：

```
L = Σᵢ log P(xᵢ | x₁, ..., xᵢ₋₁; θ)
```

等价于最小化负对数似然（即交叉熵损失）：

```
Loss = -1/N × Σᵢ log P(xᵢ | x₁, ..., xᵢ₋₁; θ)
```

训练过程（Teacher Forcing）： 1. 输入序列 $[x_1, x_2, ..., x_{n-1}]$ 2. 模型并行预测 $[x_2, x_3, ..., x_n]$ 3. 对每个位置计算交叉熵损失，取平均 4. 通过 Causal Mask 确保位置 i 只看到 ≤i 的 token

关键理解：预训练不是一个 token 一个 token 地训练，而是一次性输入整个序列，通过 Causal Mask 并行计算所有位置的 loss。这就是 Teacher Forcing 的效率优势。

精简版：

NTP 目标：给定前文预测下一个 token，最小化交叉熵。训练时用 Teacher Forcing 并行计算所有位置的预测和 loss，通过 Causal Mask 保证因果性。

Q5: 交叉熵损失函数的公式是什么？为什么用交叉熵？

详细版：

交叉熵衡量两个概率分布的差异：

```
H(p, q) = -Σ p(x) log q(x)
```

其中 p 是真实分布（one-hot），q 是模型预测的概率分布。

在 NTP 中，真实标签 y 是 one-hot 向量（只有正确 token 位置为 1），所以：

```
Loss = -log q(y_true)
```

即负对数似然，鼓励模型给正确 token 分配更高的概率。

为什么用交叉熵？ 1. **信息论基础**：最小化交叉熵等价于最小化模型分布和真实分布的 KL 散度 2. **梯度性质好**：配合 softmax 使用时，梯度就是 (预测概率 - 真实标签)，简洁高效 3. **对数损失惩罚自信的错误**：如果模型对错误 token 给了很高的概率，loss 会非常大

PyTorch 实现：

```
loss = F.cross_entropy(logits.view(-1, vocab_size), targets.view(-1))
```

`F.cross_entropy` 内部会先做 softmax 再计算交叉熵，所以输入是 logits 而非概率。

精简版：

交叉熵 Loss = -log P(正确token)，鼓励模型给正确 token 高概率。它等价于最小化 KL 散度，与 softmax 配合梯度简洁（预测-真实），且能严厉惩罚自信的错误预测。

Q6: 什么是学习率调度？为什么需要 Warmup + Cosine Annealing?

详细版：

学习率调度（Learning Rate Schedule）在训练过程中动态调整学习率。

Warmup 阶段：学习率从 0（或很小的值）线性增长到目标学习率。 - **目的**：训练初期模型参数随机，梯度方向不稳定。大学习率会导致参数剧烈震荡甚至发散。Warmup 让模型先"热身"，找到相对合理的参数空间后再加速。 - 通常 warmup 步数为总步数的 1-5%

Cosine Annealing 阶段：学习率按余弦曲线平滑衰减到最小值。

```
lr = lr_min + 0.5 * (lr_max - lr_min) * (1 + cos(π * t / T))
```

- **优势**：初期衰减慢、中期衰减快、后期衰减慢，比线性衰减更平滑

- **为什么不用线性衰减**：线性衰减中期和后期衰减速度一致，余弦衰减更符合训练动态——中期快速收敛，后期精细调整

MiniMind 的配置：warmup + cosine annealing，这是 LLM 预训练的标准配置。

精简版：

Warmup：学习率从 0 线性增到目标值，防止训练初期参数震荡。Cosine Annealing：学习率按余弦平滑衰减，初期慢→中期快→后期慢。这是 LLM 预训练的标准配方。

Q7: 什么是混合精度训练（Mixed Precision Training）？

详细版：

混合精度训练使用 FP16（半精度浮点）和 FP32（全精度浮点）混合进行训练：

为什么用 FP16？ - 显存占用减半（FP16 是 2 字节，FP32 是 4 字节） - GPU Tensor Core 对 FP16 有专门优化，计算速度翻倍 - 通信量减少（多卡训练时）

为什么不全用 FP16？ - FP16 精度有限（范围约 ±65504，精度约 3-4 位有效数字） - 梯度可能下溢（太小变成 0）或溢出

混合精度策略（PyTorch AMP）： 1. **前向传播**：FP16 计算（快） 2. **反向传播**：FP16 计算（快） 3. **权重更新**：FP32 主权重（精确） 4. **Loss Scaling**：将 loss 放大一个因子，防止梯度下溢；更新前再缩小回来

```
scaler = torch.cuda.amp.GradScaler()
with torch.cuda.amp.autocast():
    output = model(input)
    loss = criterion(output, target)
scaler.scale(loss).backward()
scaler.step(optimizer)
scaler.update()
```

精简版：

混合精度用 FP16 做前向/反向（省显存、快 2 倍），用 FP32 存权重和更新（保精度）。Loss Scaling 防止 FP16 梯度下溢。PyTorch 通过 AMP 自动管理。

Q8: 什么是梯度累积（Gradient Accumulation）？为什么需要?

详细版：

梯度累积是在多个 mini-batch 上累加梯度后才执行一次参数更新的技术。

```
for i, batch in enumerate(dataloader):
    loss = model(batch) / accumulation_steps
    loss.backward()  # 梯度累加到 .grad 上
    if (i + 1) % accumulation_steps == 0:
        optimizer.step()
        optimizer.zero_grad()
```

为什么需要？ - GPU 显存有限，无法一次装下大 batch - 等效 batch_size = micro_batch_size × accumulation_steps - LLM 训练通常需要较大的 batch_size 才能稳定

梯度累积 vs 大 batch_size： - 效果上等价（梯度是求和/平均，可以分多次算） - 区别：梯度累积期间 BN 统计量不同（但 Transformer 用 LayerNorm 不受影响） - 梯度累积的 throughput 稍低（多次前向/反向），但突破了显存限制

注意：loss 要除以 accumulation_steps，确保梯度的量级正确。

精简版：

梯度累积：多个 mini-batch 的梯度累加后才更新参数，等效增大 batch_size。用于显存不足以装下大 batch 的场景。注意 loss 要除以累积步数。

Q9: 什么是梯度裁剪（Gradient Clipping）？为什么需要?

详细版：

梯度裁剪在反向传播后、参数更新前，对梯度的范数进行截断：

```
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
```

方式：计算所有参数梯度的 L2 范数，如果超过 max_norm，则等比例缩小所有梯度：

```
if ||g|| > max_norm:
    g = g × (max_norm / ||g||)
```

为什么需要？ 1. **防止梯度爆炸**：某些 batch 可能产生异常大的梯度，导致参数一步更新过大 2. **稳定训练**：尤其在训练初期和学习率较大时 3. **LLM 训练标配**：几乎所有 LLM 训练都使用 max_norm=1.0

注意：梯度裁剪只缩放范数过大的梯度，不改变梯度方向。它和 L2 正则化不同——L2 正则化作用在参数上，梯度裁剪作用在梯度上。

精简版：

梯度裁剪：反向传播后，如果梯度 L2 范数超过阈值（通常 1.0），等比缩小所有梯度。防止梯度爆炸，稳定训练。不改变梯度方向，只限制步长。

Q10: SFT（有监督微调）和预训练有什么区别？

详细版：

特性	预训练（Pretrain）	SFT（Supervised Fine-Tuning）
目标	学习语言的通用知识	学习遵循指令的能力
数据	大量无标注文本	高质量 (指令, 回复) 对
数据量	海量（TB 级）	相对少（万到百万条）
Loss 计算	所有 token 都计算 loss	只计算 assistant 回复部分的 loss
学习率	较大	较小（通常是预训练的 1/10）
轮数	通常 1 epoch	可能多个 epoch
输出行为	续写/补全	问答/对话

核心区别： 1. **Loss Mask**：SFT 只在 assistant 回复部分计算 loss，不计算 system prompt 和 user query 的 loss 2. **数据格式**：SFT 使用 chat_template 格式化数据 3. **学习目标**：预训练学「语言能力」，SFT 学「遵循指令和对话格式」

精简版：

预训练用大量无标注文本学语言通用知识，所有 token 都计算 loss。SFT 用指令-回复对学遵循指令能力，只对 assistant 回复计算 loss。SFT 学习率更小，数据量更少。

Q11: Loss Mask 在 SFT 中是怎么实现的？为什么只计算 assistant 部分的 loss？

详细版：

为什么只计算 assistant 部分？ - 我们希望模型学习「给定指令如何回复」，而不是学习「如何生成指令」 - 如果对 user prompt 也计算 loss，模型会花费容量去学习重复用户的话，浪费模型能力 - system prompt 是固定的模板，更不需要学习

实现方式： 1. 将对话拼接成完整序列：`<system> ... <user> ... <assistant> ...` 2. 构建与输入等长的 loss_mask，assistant 部分为 1，其余为 0 3. 计算 loss 时乘以 mask：

```
# 构建 loss_mask
loss_mask = torch.zeros_like(input_ids, dtype=torch.float)
# 找到 assistant 回复的起止位置，设为 1
loss_mask[assistant_start:assistant_end] = 1.0

# 计算 masked loss
logits = model(input_ids)
loss = F.cross_entropy(logits.view(-1, vocab_size), targets.view(-1), reduction='none')
loss = (loss * loss_mask.view(-1)).sum() / loss_mask.sum()
```

多轮对话：如果有多轮对话，每轮的 assistant 回复都要标记为 1。

精简版：

Loss Mask 是与输入等长的 0/1 向量，assistant 回复部分为 1，其余为 0。计算 loss 时逐 token 乘以 mask 再求平均。只训练模型生成回复的能力，不浪费容量学习重复用户输入。

Q12: chat_template 是什么？为什么需要？

详细版：

chat_template 是将多轮对话结构化为模型输入的格式模板。不同模型有不同的 template。

典型格式（ChatML 风格）：

```
<|im_start|>system
你是一个有帮助的助手。<|im_end|>
<|im_start|>user
你好<|im_end|>
<|im_start|>assistant
你好！有什么我可以帮助你的吗？<|im_end|>
```

为什么需要 chat_template？ 1. **区分角色**：模型需要知道哪部分是 system、user、assistant 2. **标记边界**：特殊 token（如 `<|im_start|>`，`<|im_end|>`）帮助模型识别回复的起止 3. **构建 loss_mask**：根据 template 的特殊 token 定位 assistant 部分 4. **推理一致性**：推理时也要用相同的 template 格式化输入

注意：SFT 训练和推理时必须使用完全一致的 chat_template，否则模型行为不可预期。

精简版：

chat_template 用特殊 token 将对话格式化为 `<role>content</role>` 结构。区分角色、标记回复边界、帮助构建 loss_mask。训练和推理必须用同一 template。

Q13: LoRA 的原理是什么？公式推导？

详细版：

核心思想：预训练权重矩阵 W_0 的增量 ΔW 是低秩的，可以分解为两个小矩阵的乘积。

公式：

```
h = (W₀ + ΔW)x = W₀x + BAx
```

其中： - $W_0 \in R^{d\times d}$：冻结的预训练权重 - $A \in R^{r\times d}$：降维矩阵，$r << d$ - $B \in R^{d\times r}$：升维矩阵

初始化： - A 用高斯随机初始化 - B 初始化为全零 - 初始时 $BA = 0$，模型行为与原始模型一致

缩放因子：

```
h = W₀x + (α/r) · BAx
```

α 是超参数，控制 LoRA 增量的影响大小。通常 $\alpha = r$ 或 $\alpha = 2r$。

为什么有效？ - Aghajanyan et al. 发现预训练模型的权重更新具有低秩特性（intrinsic dimensionality 很低） - LoRA 通过低秩分解捕获这种结构，大幅减少可训练参数

参数量对比： - 全量微调：$d \times d = d^2$ 个参数 - LoRA：$r \times d + d \times r = 2rd$ 个参数 - 当 $r=8$, $d=768$ 时：$2\times8\times768 = 12{,}288$ vs $768^2 = 589{,}824$，减少约 98%

精简版：

LoRA：$h = W_0x + BAx$，将权重增量 ΔW 分解为低秩矩阵 $B(d\times r) \cdot A(r\times d)$。B 初始化为 0 保证初始行为不变。$r=8$ 时参数量仅为全量微调的 2%。

Q14: LoRA 的秩 r 怎么选择？

详细版：

r 的选择指南： - **r=4~8**：适用于大多数场景，尤其是任务和预训练域相近时 - **r=16~64**：任务复杂或与预训练域差距大时 - **r=1~2**：非常简单的任务或参数预算极紧

影响因素： 1. **任务复杂度**：简单任务（如情感分类）用小 r，复杂任务（如数学推理）用大 r 2. **数据量**：数据多可以用大 r（不容易过拟合），数据少用小 r 3. **模型规模**：大模型的 intrinsic dimension 更低，可以用相对更小的 r 4. **实验验证**：最终需要在验证集上对比不同 r 的效果

MiniMind 的实践：作为一个 64M 的小模型，r=8~16 通常是合理的选择。

精简版：

通常 r=4~8 即可满足大多数任务。任务越复杂/数据越多可增大 r。最终需实验验证。小模型如 MiniMind 用 r=8~16 即可。

Q15: LoRA 通常加在哪些层？为什么？

详细版：

常见选择：加在注意力层的 Q/K/V/O 投影矩阵上。

为什么加在注意力层？ 1. 注意力层决定了 token 间的信息交互模式，调整它对任务适配影响最大 2. 实验表明对 Q 和 V 加 LoRA 效果最好（原论文结论） 3. FFN 层主要存储知识，如果需要注入大量新知识则也需要加

各种配置的效果： - 只加 Q：效果不错但不是最优 - 加 Q+V：经典配置，效果好 - 加 Q+K+V+O：更彻底，参数略多 - 加全部层（含 FFN）：参数最多但某些任务效果最好

精简版：

通常加在注意力层的 Q/V 投影上（经典配置）。注意力层决定 token 交互模式，调整影响大。如需注入大量新知识可扩展到 FFN。

Q16: DPO 的损失函数是什么？请推导。

详细版：

背景：DPO（Direct Preference Optimization）直接从偏好数据中学习策略，无需单独训练奖励模型。

偏好数据格式：给定 prompt x，有一个 chosen 回复 y_w（人类偏好的）和一个 rejected 回复 y_l。

DPO 损失：

```
L_DPO = -E[log σ(β · (log π_θ(y_w|x)/π_ref(y_w|x) - log π_θ(y_l|x)/π_ref(y_l|x)))]
```

简化：

```
L_DPO = -E[log σ(β · (r_w - r_l))]
其中 r = log π_θ(y|x) - log π_ref(y|x) 是隐式奖励
```

推导直觉： 1. RLHF 的目标是最大化奖励模型的分数同时不偏离参考策略太远（KL 约束） 2. 从 KL 约束的优化问题出发，可以解析地得到最优策略 3. 将奖励模型替换为策略和参考策略的对数概率比，消除了奖励模型 4. 最终只需一个二分类目标：让 chosen 的隐式奖励高于 rejected

β 的作用： - β 控制对参考策略的偏离程度 - β 越大，越保守（不偏离参考策略太远） - β 越小，越激进（更多地优化偏好） - 通常 β ∈ [0.1, 0.5]

精简版：

DPO 损失 = -log σ(β·(chosen 的隐式奖励 - rejected 的隐式奖励))。隐式奖励 = log π_θ(y|x) - log π_ref(y|x)。β 控制偏离参考策略的程度。直接从偏好数据学习，无需奖励模型。

Q17: DPO 和 PPO 有什么区别？各有什么优缺点？

详细版：

特性	DPO	PPO
是否需要奖励模型	不需要	需要
训练复杂度	简单（类似 SFT）	复杂（4 个模型交互）
内存占用	需要 policy + reference 两个模型	需要 policy + reference + reward + value 四个模型
训练稳定性	较好	需要精心调参
偏好建模	隐式（通过策略和参考策略的比值）	显式（通过奖励模型打分）

特性	DPO	PPO
在线/离线	离线（直接用偏好数据）	在线（需要采样新数据）
效果	在很多任务上接近 PPO	理论上更强（在线学习）

DPO 优势：实现简单、训练快、稳定、不需要奖励模型 **DPO 劣势**：离线学习有分布偏移问题、无法利用奖励模型的泛化能力

PPO 优势：在线学习、奖励模型可泛化 **PPO 劣势**：需要 4 个模型、训练不稳定、超参敏感

精简版：

DPO 直接从偏好数据学习，不需要奖励模型，实现简单稳定。PPO 需要训练奖励模型，是在线学习，理论更强但需要 4 个模型且训练复杂。DPO 是 PPO 的简化替代方案。

Q18: PPO 中的 clip 机制是什么？为什么需要？

详细版：

PPO（Proximal Policy Optimization）的核心是限制策略更新的幅度。

PPO-Clip 目标：

```
L_CLIP = E[min(r(θ)·A, clip(r(θ), 1-ε, 1+ε)·A)]
```

其中： - r(θ) = π_θ(a|s) / π_old(a|s)：新旧策略的概率比 - A：优势函数（Advantage） - ε：裁剪范围，通常 0.1~0.2

clip 的作用： - 当 r(θ) > 1+ε 且 A > 0 时：策略在往好的方向走，但已经走太远了，clip 阻止继续走 - 当 r(θ) < 1-ε 且 A < 0 时：策略在远离坏的方向，但已经走太远了，clip 阻止 - 总之：每次更新不会让新策略偏离旧策略太多

为什么需要？ - 策略梯度方法如果更新幅度太大，性能可能突然崩塌 - TRPO 用 KL 散度约束，但计算昂贵 - PPO 用 clip 近似实现 trust region，简单高效

精简版：

PPO 的 clip 将策略概率比限制在 [1-ε, 1+ε] 范围内，防止单次更新偏离旧策略太多。是 trust region 思想的简化实现，平衡了训练效率和稳定性。

Q19: GRPO 是什么？和 PPO 有什么区别？

详细版：

GRPO（Group Relative Policy Optimization）是 DeepSeek 提出的改进方法：

核心思想：不需要单独的 value model（价值网络），而是通过组内相对比较来估计优势函数。

具体做法： 1. 对同一个 prompt，采样 G 个回复 $\{y_1, y_2, ..., y_G\}$ 2. 用奖励模型对每个回复打分 $\{r_1, r_2, ..., r_G\}$ 3. 组内归一化得到优势估计：$A_i = (r_i - mean(r)) / std(r)$ 4. 用归一化后的优势函数做类似 PPO 的策略优化

与 PPO 的区别：

特性	PPO	GRPO
Value Model	需要	不需要
优势估计	GAE（需要 value model）	组内归一化奖励
内存	4 个模型	3 个模型（少了 value model）
采样	每个 prompt 采样 1 个	每个 prompt 采样 G 个

精简版：

GRPO 去掉了 PPO 的 value model，通过对同一 prompt 采样多个回复并在组内归一化奖励来估计优势函数。减少了内存（3 个模型 vs PPO 的 4 个），训练更简单。

Q20: 什么是知识蒸馏？黑盒蒸馏和白盒蒸馏有什么区别？

详细版：

知识蒸馏（Knowledge Distillation）将大模型（Teacher）的知识转移到小模型（Student）。

黑盒蒸馏：只能获取 Teacher 的输出概率分布，不能访问内部结构。

```
L = α · KL(softmax(z_s/T), softmax(z_t/T)) + (1-α) · CE(z_s, y_hard)
```

- z_s：Student 的 logits
- z_t：Teacher 的 logits
- T：温度参数
- y_hard：真实标签

- α：两种 loss 的权重

白盒蒸馏：可以访问 Teacher 的中间层表示。 - 除了输出分布匹配外，还可以做中间层特征对齐 - 例如：让 Student 的某些层输出与 Teacher 对应层的输出接近 - 需要 Teacher 和 Student 有类似的结构

MiniMind 的蒸馏方案： - 从 Qwen 系列蒸馏到 MiniMind - 主要采用黑盒蒸馏（软标签 KL 散度）

精简版：

知识蒸馏让小模型学习大模型的输出。黑盒蒸馏只用 Teacher 的输出概率（KL 散度），白盒蒸馏还可以对齐中间层特征。MiniMind 从 Qwen 做黑盒蒸馏。

Q21: 蒸馏中温度 T 的作用是什么？

详细版：

蒸馏中的温度参数 T 用于软化 Teacher 模型的输出分布：

```
softmax(z_i / T) = exp(z_i / T) / Σ exp(z_j / T)
```

T 的影响： - **T=1**：标准 softmax，概率分布尖锐（接近 one-hot） - **T→∞**：趋近均匀分布，所有 token 概率相近 - **T 适中（2-10）**：软化分布，暴露 Teacher 对各 token 的相对偏好（暗知识）

为什么需要软化？ - 标准 softmax 下，Teacher 的输出高度集中在正确答案上 - 其他 token 的概率极小，Student 学不到它们之间的相对关系 - 温度升高后，"第二选择"、"第三选择"的概率被放大，包含了 Teacher 学到的更细粒度的知识 - 例如：Teacher 预测 "猫" 的概率 0.9，"狗" 0.05，"鱼" 0.01。软化后变为 "猫" 0.5，"狗" 0.25，"鱼" 0.15，Student 能学到"猫"和"狗"比"鱼"更相关

注意：Teacher 和 Student 的 softmax 必须使用相同的 T。

精简版：

温度 T 软化 Teacher 的输出分布，让 Student 学到 token 之间的相对关系（暗知识），而非只学到最高概率的正确答案。T 越大分布越均匀，通常 T=2~10。

Q22: KL 散度是什么？为什么在蒸馏和 RLHF 中大量使用？

详细版：

KL 散度（Kullback-Leibler Divergence）衡量两个概率分布之间的"距离"：

```
KL(P || Q) = Σ P(x) log(P(x) / Q(x))
```

性质：- KL(P||Q) ≥ 0，当且仅当 P = Q 时等于 0 - 不对称：KL(P||Q) ≠ KL(Q||P) - 不是真正的距离度量

在蒸馏中：KL(Teacher || Student)，让 Student 的分布接近 Teacher **在 RLHF/DPO 中**：KL(π_θ || π_ref)，约束训练后的策略不偏离参考策略太远

为什么用 KL 而不是其他度量？ 1. 信息论基础扎实，有明确的优化含义 2. 梯度计算高效 3. 在概率分布比较中有天然优势 4. 与交叉熵关系：H(P,Q) = H(P) + KL(P||Q)，最小化交叉熵等价于最小化 KL

前向 KL vs 逆向 KL：- KL(P||Q)（前向）：Q 倾向于覆盖 P 的所有模式（mode covering） - KL(Q||P)（逆向）：Q 倾向于集中在 P 的某个模式上（mode seeking）

精简版：

KL(P||Q) = Σ P(x)log(P(x)/Q(x))，衡量两个分布的"距离"，非负且不对称。蒸馏中让 Student 逼近 Teacher 分布，RLHF 中约束策略不偏离参考太远。

Q23: 训练过程中如何判断和处理过拟合？

详细版：

判断过拟合的信号：1. 训练 loss 持续下降但验证 loss 开始上升 2. 验证集指标不再提升 3. 模型在训练集上表现完美但在新数据上效果差 4. 生成的文本开始大段复制训练数据

防止过拟合的方法：1. **数据层面**：增加数据量、数据增强、提高数据多样性 2. **正则化**：Dropout、Weight Decay（L2 正则） 3. **训练策略**：早停（Early Stopping）、减小学习率、减少训练轮数 4. **模型层面**：减小模型规模、使用 LoRA 等参数高效方法 5. **权重共享**：MiniMind 的 embed/lm_head 共享也是隐式正则化

小模型更容易过拟合：MiniMind 只有 64M 参数，在大量数据上不容易过拟合，但在小数据集上需要注意。SFT 阶段通常数据量较少，更需要关注。

精简版：

过拟合：训练 loss 下降但验证 loss 上升。应对：增加数据、Dropout、Weight Decay、早停、减小学习率。SFT 阶段数据少时尤其要注意。

Q24: 数据质量对 LLM 训练有多重要?

详细版：

"数据质量 > 数据数量" 在 LLM 训练中已成为共识：

预训练阶段： - 垃圾数据（广告、乱码、重复内容）会降低模型质量 - 需要大量数据清洗：去重、过滤低质量、语言检测、毒性过滤 - Scaling Law 假设的是高质量数据

SFT 阶段： - LIMA 论文证明：1000 条高质量 SFT 数据就能显著提升对话能力 - 低质量 SFT 数据可能让模型学到错误的回复模式 - 质量 >> 数量

偏好对齐阶段（DPO/PPO）： - 偏好数据的标注一致性很重要 - 模糊的偏好标注会让模型学到矛盾的目标

MiniMind 实践： - 预训练数据经过了去重和清洗 - SFT 数据选择了高质量的多轮对话数据 - 小模型对数据质量更敏感（模型容量小，更容易被噪声数据污染）

精简版：

数据质量 > 数据数量。预训练需要去重和过滤；SFT 1000 条高质量数据即可显著提升；偏好对齐需要一致性强的标注。小模型对数据质量更敏感。

Q25: 什么是断点续训？如何实现?

详细版：

断点续训（Resume Training / Checkpoint Recovery）是在训练中断后从上次保存的状态继续训练。

需要保存的内容： 1. **模型权重**：model.state_dict() 2. **优化器状态**：optimizer.state_dict()（包含动量、二阶矩估计等） 3. **学习率调度器状态**：scheduler.state_dict() 4. **当前 epoch/step**：知道从哪里继续 5. **随机种子状态**：torch/numpy/python 的 random state（保证数据加载顺序一致） 6. **GradScaler 状态**：如果用了混合精度

```
# 保存
torch.save({
    'model': model.state_dict(),
```

```
    'optimizer': optimizer.state_dict(),
    'scheduler': scheduler.state_dict(),
    'step': global_step,
    'scaler': scaler.state_dict(),
}, 'checkpoint.pt')

# 恢复
ckpt = torch.load('checkpoint.pt')
model.load_state_dict(ckpt['model'])
optimizer.load_state_dict(ckpt['optimizer'])
scheduler.load_state_dict(ckpt['scheduler'])
scaler.load_state_dict(ckpt['scaler'])
```

为什么需要保存优化器状态？ - Adam 优化器维护了一阶动量和二阶动量的指数移动平均 - 如果不恢复，这些统计量从零开始，会导致训练不稳定（类似重新 warmup）

精简版：

断点续训需要保存并恢复：模型权重、优化器状态（Adam 的动量）、调度器状态、当前 step、随机种子。只恢复模型权重而不恢复优化器状态会导致训练不稳定。

Q26: DDP（Distributed Data Parallel）多卡训练的原理是什么？

详细版：

DDP 核心思想：数据并行。每张 GPU 持有完整的模型副本，各自处理不同的数据子集，梯度同步后统一更新。

工作流程： 1. 每张 GPU 创建模型的完整副本 2. 数据集通过 DistributedSampler 分割，每张 GPU 获取不同的 mini-batch 3. 每张 GPU 独立做前向和反向传播 4. 反向传播时，通过 **AllReduce** 操作同步所有 GPU 的梯度（求平均） 5. 每张 GPU 用同步后的梯度更新自己的参数（因为梯度一致、模型初始化一致，参数保持同步）

关键技术： - **Ring-AllReduce**：高效的梯度同步通信算法，通信量与 GPU 数量无关 - **Gradient Bucketing**：将梯度分桶，在反向传播过程中就开始通信（overlap computation and communication）

DDP vs DP： - DP（DataParallel）：单进程多线程，有 GIL 限制，梯度汇聚到一张卡 - DDP：多进程，通信更高效，负载均衡，是推荐方案

精简版：

DDP：每张 GPU 持有完整模型，处理不同数据，梯度通过 AllReduce 同步后统一更新。相比 DP（单进程）更高效，通过 Ring-AllReduce 和梯度桶化优化通信。

Q27: AdamW 优化器的原理是什么？为什么 LLM 训练普遍使用 AdamW?

详细版：

Adam（Adaptive Moment Estimation）维护两个指数移动平均：

```
m_t = β₁ · m_{t-1} + (1-β₁) · g_t       # 一阶动量（梯度均值）
v_t = β₂ · v_{t-1} + (1-β₂) · g_t²       # 二阶动量（梯度方差）
m̂_t = m_t / (1-β₁^t)                     # 偏差修正
v̂_t = v_t / (1-β₂^t)
θ_t = θ_{t-1} - lr · m̂_t / (√v̂_t + ε)
```

AdamW（Adam with Weight Decay）：将 weight decay 从梯度中解耦出来：

```
θ_t = θ_{t-1} - lr · (m̂_t / (√v̂_t + ε) + λ · θ_{t-1})
```

原始 Adam 的 L2 正则化和自适应学习率耦合，效果不正确。AdamW 直接在权重上做衰减，是正确的实现。

为什么 LLM 普遍用 AdamW? 1. 自适应学习率，不同参数用不同的有效学习率 2. 动量平滑，减少梯度噪声的影响 3. Weight Decay 正则化防止过拟合 4. 对超参数不太敏感，β_1=0.9, β_2=0.999 几乎是通用设置

精简版：

AdamW 维护梯度的一阶和二阶动量，自适应调整每个参数的学习率，并正确解耦了 weight decay。LLM 普遍使用因为收敛快、对超参不敏感。

Q28: SFT 数据中多轮对话是怎么处理的?

详细版：

多轮对话需要将多轮上下文拼接为一个完整序列，并正确构建 loss_mask：

```
<|im_start|>system
你是一个助手<|im_end|>
```

```
<|im_start|>user
你好<|im_end|>
<|im_start|>assistant
你好！有什么可以帮你的？<|im_end|>     ← loss_mask=1
<|im_start|>user
今天天气怎么样<|im_end|>
<|im_start|>assistant
今天天气晴朗。<|im_end|>              ← loss_mask=1
```

处理要点： 1. 所有轮次拼接为一个序列，通过特殊 token 分隔角色 2. loss_mask 只标记所有 assistant 回复部分 3. 如果总长度超过 max_length，需要截断（通常从头部截断，保留最近的上下文） 4. 后续轮次的 assistant 可以看到前面所有轮次的内容（因为 Causal Mask）

精简版：

多轮对话拼接为一个序列，用特殊 token 分隔。loss_mask 标记所有轮次的 assistant 回复。超长截断时通常保留最近的上下文。

Q29: 预训练数据的预处理流程是什么样的?

详细版：

典型的预训练数据预处理流程：

1. **数据收集**：从互联网、书籍、代码库等多来源收集
2. **去重**：
3. 精确去重：哈希去重
4. 模糊去重：MinHash + LSH，去除近似重复
5. **质量过滤**：
6. 语言检测，过滤非目标语言
7. Perplexity 过滤（用小语言模型打分，去掉 PPL 过高/过低的）
8. 启发式规则：过滤过短/过长、特殊字符比例过高、重复率过高的文本
9. **安全过滤**：去除有毒内容、个人信息
10. **Tokenize**：将文本用 Tokenizer 编码为 token 序列
11. **拼接和分块**：将多个文档拼接，按 max_length 切分为训练样本

精简版：

预训练数据流程：收集 → 去重（哈希/MinHash）→ 质量过滤（语言/PPL/规则）→ 安全过滤 → Tokenize → 拼接分块。

Q30: 权重衰减（Weight Decay）的作用是什么？哪些参数不应该加 Weight Decay?

详细版：

Weight Decay 在每次更新时对权重施加衰减：θ = θ - lr·λ·θ，等价于 L2 正则化。

作用： - 防止权重过大，起到正则化效果 - 让模型倾向于使用更小的权重，提高泛化能力

不应加 Weight Decay 的参数： 1. **偏置项（bias）**：通常不正则化，它们调整激活的偏移，数量少不会导致过拟合 2. **归一化层参数**：RMSNorm/LayerNorm 的 γ（和 β），它们的作用是缩放和偏移，正则化会干扰归一化效果 3. **Embedding 层**：有些实现中也不对 Embedding 加 Weight Decay

```
param_groups = [
    {'params': decay_params, 'weight_decay': 0.1},
    {'params': no_decay_params, 'weight_decay': 0.0},
]
optimizer = torch.optim.AdamW(param_groups, lr=lr)
```

精简版：

Weight Decay 对权重做衰减起正则化效果。bias、LayerNorm/RMSNorm 参数通常不加 Weight Decay，因为正则化会干扰它们的功能。

Q31: 训练中 Loss 突然飙升（Loss Spike）是什么原因？怎么处理?

详细版：

可能原因： 1. **学习率过大**：参数更新幅度过大，跳出了良好的参数空间 2. **数据问题**：遇到异常样本（过长、特殊编码、数据损坏） 3. **梯度爆炸**：某些层的梯度突然变大 4. **混合精度溢出**：FP16 数值范围有限 5. **BF16 不稳定**：某些操作在 BF16 下精度不足

处理方法： 1. **梯度裁剪**：限制梯度范数 2. **检查数据**：定位并修复异常样本 3. **降低学习率**：尤其在训练后期 4. **增加 warmup**：训练初期更平稳 5. **Loss Scaling**：混合精度训练时动态调整 6. **跳过异常 batch**：检测到 loss 异常时跳过该 batch

精简版：

Loss Spike 常因学习率过大、异常数据、梯度爆炸或混合精度溢出。应对：梯度裁剪、检查数据、降低学习率、增加 warmup、动态 Loss Scaling。

Q32: 什么是奖励模型（Reward Model）？如何训练?

详细版：

奖励模型将模型的输出映射为一个标量分数，表示该输出的质量。

训练数据：偏好对（chosen, rejected），表示人类偏好 chosen 优于 rejected。

训练目标（Bradley-Terry 模型）：

```
L = -E[log σ(r(x, y_w) - r(x, y_l))]
```

让 chosen 回复的分数高于 rejected 回复的分数。

模型结构：通常在 LLM 基础上，将最后的 lm_head 替换为一个输出标量分数的线性层：

```
reward = Linear(d_model, 1)(last_hidden_state)
```

应用： - PPO/GRPO 中为模型生成的回复打分 - 用于 Best-of-N 采样（生成 N 个回复，选分数最高的） - 数据筛选

精简版：

奖励模型将输出映射为标量分数。用偏好对训练：L = -log σ(r(chosen) - r(rejected))。结构是 LLM 最后一层接线性投影到 1 维。用于 PPO/GRPO 的奖励信号。